THREAT-MODEL

marrow / ARI — Threat Model

Scope: the `marrow` reference runtime and the security properties an ARI-conformant runtime should provide. **Method:** STRIDE over the real attack surface, grounded in the current code (`src/core/`, `src/bindings/`, `python/marrow/`). **Status:** living document for a pre-1.0 project. This is a *static* review, not a substitute for an independent audit, fuzzing campaign, or penetration test of a deployed system.

---

1. What marrow is (and isn't), security-wise

marrow is a **library**, not a service or a sandbox. It runs an agent loop in your process. Two consequences drive everything below:

1. **Registered tool bodies execute with full host privileges.** marrow does not and cannot sandbox a tool you register. If you register a tool that calls `subprocess.run`, and a prompt-injected model invokes it, that is your design, not a runtime vulnerability.
2. **The dominant real-world risk is prompt-injection → tool-call.** The model is untrusted output. The runtime's job is to *bound the blast radius* (caps, redaction, cancellation, backpressure) and to give you the primitives to apply least privilege — not to make arbitrary tools safe.

Trust boundaries

Zone	Examples	Treatment
Untrusted	LLM completions, tool-call args, tool results, message `content`/`metadata` from outside, a `StateStore` DB shared with other writers	Validate, cap, redact, never `eval`
Trusted (by configuration)	the host process, registered tool bodies, provider `base_url`, API keys, embedding application code	Out of scope to defend *from*; in scope to avoid *leaking*

---

2. Attack surface

Surface	Entry point	Notes

Message ingress	`Message.make`, `.content` setter, router `send`	size caps, role set
Tool dispatch	`ToolRegistry.invoke`, `ToolBox` adapter	JSON in/out, size caps, error redaction
Provider egress	`OpenAI/Anthropic/Ollama` providers	network, API keys, `base_url`, timeouts
Multi-agent routing	`AgentRouter` inboxes	backpressure / memory
Persistence	`InMemory/SQLiteStateStore`	at-rest content, round-trip integrity
Concurrency	C++ `shared_mutex`, GIL release	data races, UB
Supply chain	build + optional deps	provenance, advisories

---

3. Findings (STRIDE)

Severity is qualitative (relative to a library at this stage). "Status" reflects this commit.

ID	STRIDE	Finding	Severity	
T1	DoS	Providers ignored `timeout_ms`/`cancel_token`; a hung provider call blocks an async worker thread indefinitely (8-thread pool → full stall).	Medium	**Fixed** — `Agen` OpenAI/Anthr timeout and r
T2	Tampering / DoS	4 MiB `Message.content` cap enforced in `make()` but bypassable via direct `.content =` assignment.	Medium	**Fixed** — bind
T3	Info disclosure	Tool error text could leak secrets/paths; original redactor matched only a few prefixes.	Low–Med	**Mitigated** — (OpenAI/Anth URLs) + path returning exc `strict` for unt
T4	DoS	`AgentRouter` inboxes default to **unbounded** (`max==0`); a producer with no consumer exhausts memory.	Medium	**Documented** available; AR test asserts b
T5	Info disclosure	Conversation history is stored **plaintext** by `SQLiteStateStore`/in-memory.	Low	**Documented** responsibility `StateStore`); t
T6	DoS	`RetryPolicy` retries broadly, amplifying load on non-transient failures.	Low	**Documented** `ValueError` for surprise calle the call site f
T7	Elev. of privilege	No per-agent tool ACL: any agent in a `Runtime` can invoke any registered tool.	Low (design)	**Documented** future work. I domain.
T8	Spoofing / SSRF	Provider `base_url` (Ollama, OpenAI) is caller-trusted; an attacker-controlled URL exfiltrates prompts **and** API key.	Low–Med	**Documented** never from m

T9	Tampering	`StateStore.load` reads an arbitrary-size row fully into memory.	Low	**Documented** isolate it.
T10	Supply chain	No CodeQL / dependency-audit / SBOM / signed releases.	Medium (for a standard)	**Partially add** + `pip-audit`); recommended
T11	Info disclosure / integrity	`Message.metadata` was dropped on persistence load (lossy round-trip; trace metadata loss).	Low	**Fixed** — meta

Memory safety & concurrency (C++ core)

- Access is guarded by `std::shared_mutex` consistently; `Engine::create_agent` documents its lock ordering to avoid a `create`/`send` race and deadlock.
- A ThreadSanitizer CI job exists (informational). **Recommended before 1.0:** an AddressSanitizer build in CI and a libFuzzer/`atheris` harness over tool-arg JSON parsing and message ingestion. Tracked as future work — not yet present.

---

4. Residual risk (accepted / host-owned)

- **Prompt-injection driving tool calls** — inherent to agents. Mitigate at the host: least-privilege tools, per-`Runtime` isolation, `ToolBox(strict=True)`, human-in-the-loop for dangerous actions. marrow bounds blast radius; it does not eliminate this class.
- **Compromised dependency** in an optional provider extra — mitigated by audits
  - pinning; not eliminated.
- **At-rest disk compromise** — host's encryption responsibility.

---

5. ARI-Embedded security profile

For runtimes claiming **ARI-Embedded** (robots/edge/native), these are MUSTs, not SHOULDs, because the deployments are long-running, physically exposed, and often unattended:

1. **Bounded memory** — finite inbox bounds and fixed content/tool I/O caps by default (mitigates T4).
2. **Strict redaction** — `strict`-mode error handling by default (mitigates T3).
3. **Mandatory cancellation + wall-clock timeouts** on every provider/tool call (mitigates T1), thread-safe across the control loop.
4. **No hot-path allocation when observability is disabled** (the null trace sink).
5. **Declared, near-zero dependency footprint** (smaller attack surface).

---

6. Reporting

See SECURITY.md for private disclosure. Do not file public issues for vulnerabilities.